

CS3106L Operating Systems Laboratory

Lab 5: Concurrency, Locks, and Sleep/Wakeup

Department of Computer Science and Engineering, IIT Guwahati

Complete the practicum in a scratch copy or scratch Git branch of the completed Lab 4 LUIT tree.

Central idea:

observe a race → understand atomic locking → observe blocking → explain lost-wakeup avoidance

This practicum does not ask you to redesign the scheduler or reimplement synchronization primitives. The purpose is to connect the concurrency mechanisms discussed in theory to working code: first with a controlled host-side race, then with LUIT's existing spinlocks and pipe sleep/wakeup path.

1 What you should be able to explain at the end

After completing the practicum, you should be able to explain all of the following without memorizing a code listing:

1. why a read-modify-write such as `counter++` is not automatically atomic;
2. how a legal interleaving can lose an update;
3. why a lock implementation itself needs an atomic read-modify-write operation;
4. how LUIT's `acquire()` and `release()` provide cross-hart mutual exclusion;
5. why `push_off()` / `pop_off()` solve a different problem from the atomic lock word;
6. why nested interrupt-disable state must be tracked;
7. why spinning is appropriate only for short waits;
8. how a bounded pipe creates a real condition on which a process may have to sleep;
9. what a sleep channel means in LUIT;
10. why `sleep(chan, lock)` takes the lock as an argument;
11. how an unlock-then-sleep gap can lose a wakeup; and
12. why a woken process must re-check the shared predicate.

2 Scope and safety rules

Use a scratch copy or scratch Git branch of the completed Lab 4 tree. Preserve all earlier work.

For this practicum:

- Do **not** modify `kernel/spinlock.c`.
- Do **not** modify the scheduler or context-switch code.
- Do **not** modify `kernel/proc.c`.
- Do **not** add or renumber system calls.
- Do **not** edit generated syscall files.
- Do **not** make the pipe larger merely to avoid blocking.
- Do **not** add arbitrary delays as a synchronization fix.

Only one LUIT source file is added by this package: `user/pipe_stress.c`. The host-side race demonstration is built and run on Linux, outside LUIT.

3 Part 0 - Establish a clean baseline

From the root of the completed Lab 4 tree:

```
make clean
make
make grade
```

Do not proceed if the completed tree no longer passes the regression suite that passed before this practicum. Fix the existing tree first rather than debugging concurrency on top of a broken baseline.

Copy the preflight checker and LUIT demonstration program from this package:

```
cp <PACKAGE>/tools/lab05_preflight.py tools/
cp <PACKAGE>/user/pipe_stress.c user/
python3 tools/lab05_preflight.py
```

The checker verifies the source-tree assumptions used by the practicum: the syscall-table source of truth, the existing calls used by pipe_stress, the LUIT spinlock primitives, sleep/wakeup symbols, and pipe read/write paths.

The final line must be:

```
LAB5 preflight: PASS
```

If the checker fails, read the exact diagnostic. Do not patch a missing mechanism merely to make the checker green.

4 Part A - Reproduce a lost update on the Linux host

The first experiment deliberately stays outside the kernel. It isolates one idea: a compound update can be incorrect even when each individual load and store is well defined.

Change to the package directory and build:

```
gcc -O2 -Wall -Wextra -pthread -std=c11 \
    host/race_demo.c -o race_demo
```

Run the intentionally unsafe version:

```
./race_demo unsafe 4 200000
```

A typical run looks like:

```
mode=unsafe threads=4 increments/thread=200000
expected=800000 actual=<smaller value> lost=<positive value>
OBSERVED: lost update(s)
```

The exact wrong value is not prescribed. Concurrency schedules are nondeterministic. If one run happens to produce the expected value, repeat the program:

```
for i in 1 2 3 4 5; do ./race_demo unsafe 4 200000; done
```

Now run the protected version:

```
./race_demo mutex 4 200000
```

This run must end with:

```
expected=800000 actual=800000 lost=0
PASS: mutex protected the compound update
```

Reason about one failure

Write down one legal interleaving for two increments starting from 20:

```
Thread/Hart A Thread/Hart B
load old value -> 20 load old value -> 20
compute 21 compute 21
store 21 store 21
```

Two increments were requested, but the shared value ends at 21. The failure is not that addition is mathematically wrong. The failure is that the compound operation *load – modify – store* was allowed to overlap another compound operation.

Checkpoint A. Be able to answer: Why would declaring the shared variable `volatile` not turn this compound update into an atomic operation?

5 Part B - Read the LUIT spinlock before using it

Return to the LUIT tree. Open `kernel/spinlock.c`. Locate `acquire()`, `release()`, `push_off()`, `pop_off()`, and `holding()`.

In the course LUIT implementation, the important conceptual path is:

```
acquire(lock)
  push_off() local interrupt discipline
  reject self-acquire avoid silent self-deadlock
  atomic read-modify-write loop cross-hart ownership
  memory-ordering barrier
  record owner metadata

release(lock)
  verify ownership
  clear owner metadata
  memory-ordering barrier
  atomically release lock word
  pop_off() restore nested interrupt state
```

Do not confuse the two jobs:

Mechanism	What it prevents
atomic lock operation	two harts entering the same critical section
<code>push_off()/pop_off()</code>	same-hart interrupt re-entry causing self-deadlock while a kernel spinlock is held

5.1 Why a plain flag is not a lock

The following is incorrect:

```
while (lk->locked)
;
lk->locked = 1;
```

Construct this execution:

```
Hart 0 Hart 1
read locked -> 0 read locked -> 0
write locked = 1 write locked = 1
enter critical section enter critical section
```

Both harts pass the test. Therefore the transition from “observed free” to “claimed” must be atomic.

5.2 Inspect the generated RISC-V code

Build LUIT, then inspect the generated code for `acquire()`:

```
make
riscv64-unknown-elf-objdump -d kernel.elf | \
    sed -n '/<acquire>:/,+45p'
```

Locate the instruction sequence that implements the compiler atomic operation. The exact lowering depends on the compiler and toolchain; it may use an AMO or an LR/SC loop. Do not expect one required mnemonic. The architectural requirement is an atomic read-modify-write with appropriate ordering.

Checkpoint B. Identify which source statement provides ownership and which statements/operations provide ordering. Explain why these are related but not identical properties.

6 Part C - Integrate the LUIT pipe stress program

The package supplies `user/pipe_stress.c`. It uses only system calls that already exist in LUIT. It does not introduce a new kernel ABI.

Open the top-level Makefile and find `UPROGS`. Add `user/pipe_stress` following the exact syntax already used for the other user binaries. Do not edit generated syscall files.

Rebuild and re-run the baseline:

```
make clean
make
make grade
```

The existing regression suite must still pass.

6.1 What the program does

The program creates one pipe and four writer processes. Each writer sends exactly 2048 one-byte tokens. Therefore:

$$4 \times 2048 = 8192 \text{ bytes}$$

The parent closes its own write descriptor, waits briefly, and then drains the pipe. The delay is not a synchronization solution; it is only an observation aid that gives writers an opportunity to fill a bounded pipe and enter the existing kernel blocking path before the reader starts draining it.

The reader counts every token and validates both the total and the contribution from each writer. The final marker must be:

```
PIPE_STRESS: PASS
```

6.2 Run with one hart

Boot:

```
make qemu CPUS=1
```

At the LUIT shell:

```
pipe_stress
```

A correct final summary is:

```
pipe_stress: received=8192 expected=8192
pipe_stress: writer 0 count=2048 expected=2048
pipe_stress: writer 1 count=2048 expected=2048
pipe_stress: writer 2 count=2048 expected=2048
pipe_stress: writer 3 count=2048 expected=2048
PIPE_STRESS: PASS
```

Exit QEMU using the normal repository/QEMU exit sequence before changing the hart count.

6.3 Repeat with two and four harts

Run separate clean boots:

```
make qemu CPUS=2
# run: pipe_stress

make qemu CPUS=4
# run: pipe_stress
```

The ordering of process execution is free to change. The validated byte counts must not change.

Checkpoint C. Explain why correctness cannot depend on one particular interleaving of the four writers.

7 Part D - Find the real condition and lock in the pipe code

Do not assume a file name. Locate the code in the current tree:

```
grep -RIIn 'pipewrite\|piperead' kernel
grep -RIIn '^[:space:]*sleep[:space:]*(' kernel
grep -RIIn '^[:space:]*wakeup[:space:]*(' kernel
```

Open the matching implementation and identify the shared pipe state. The exact field names are release-specific, but the logic must answer these questions:

1. What state tells a writer that the bounded pipe has no room?
2. What state tells a reader that there is no data yet?
3. Which lock protects those fields?
4. On what channel/key does a blocked writer sleep?
5. On what channel/key does a blocked reader sleep?
6. After which state change does the writer wake readers?
7. After which state change does the reader wake writers?
8. Why is the wait condition re-checked after a wakeup?

Do not confuse a *sleep channel* with a pipe that carries bytes. A channel is a wait key used to associate sleepers with an event/condition whose state may have changed.

8 Part E - Observe blocking and waking in GDB

Use the compact reference in docs/lab05_gdb.txt while doing this part.

First locate the symbols in the current source tree as shown in Part D. Then use two terminals.

Terminal A

```
make qemu-gdb CPUS=2
```

Terminal B

```
gdb-multiarch kernel.elf
```

The repository's `.gdbinit` normally connects to QEMU. If it is not auto-loaded, use:

```
target remote localhost:1234
symbol-file kernel.elf
```

Set breakpoints on the real symbols that you located. Typical symbol names are:

```
b sleep
b wakeup
c
```

At the LUIT shell, run:

```
pipe_stress
```

When a process reaches `sleep()`, inspect the call stack and confirmed arguments/locals:

```
bt
info args
info locals
list
p cpus[$tp].proc->pid
p cpus[$tp].proc->state
p cpus[$tp].proc->chan
```

Exact local-variable names may differ across releases. Never guess a local name in GDB: use `info args` and `info locals` first.

The important observation is the control-flow relationship:

```
writer sees pipe full
  |
  v
sleep(channel, pipe-lock)
  |
process becomes SLEEPING
  |
reader removes data and changes pipe state
  |
  v
wakeup(channel)
  |
process becomes eligible to run again
  |
writer re-checks the full/not-full predicate
```

Disable or delete a breakpoint after you have observed the intended event; otherwise a stress test may stop many times.

Checkpoint D. Explain why `wakeup()` means “the state may now permit progress” rather than “the condition is guaranteed true when this process next runs.”

9 Part F - Reconstruct the lost-wakeup bug

Consider this deliberately broken waiting shape:

```
acquire(&lock);
if (!condition) {
    release(&lock);
    sleep_somewhat();
}
```

Construct the following schedule:

```
waiter  signaler
-----
acquire(lock)
check condition -> false
release(lock)

                                acquire(lock)
                                make condition true
                                wakeup(...)
                                release(lock)

sleep_somewhat()
```

The wakeup occurred before the waiter actually became asleep. If no later state change generates another wakeup, the waiter can remain asleep forever.

Now compare the LUIT-style pattern:

```
acquire(&lock);
while (!condition)
    sleep(chan, &lock);
/* lock is held again here */
release(&lock);
```

The lock argument lets the kernel coordinate releasing the condition lock with the transition to the sleeping state so that the signaler cannot slip through the dangerous gap.

The while is equally important. A wakeup says only that relevant shared state changed. Before proceeding, the woken execution must regain the lock and check whether the predicate is actually true for it.

Checkpoint E. Give one reason that a process can wake and later find the predicate false again. Examples include another runnable process consuming the newly available resource first, or a broadcast-style wake that wakes more than one waiter.

10 Part G - Explain why LUIT disables local interrupts around spinlocks

Cross-hart exclusion and local interrupt control solve different problems.

Suppose Hart 0 executes:

```
acquire(&L);
... kernel critical section ...
```

If an interrupt were allowed to arrive on the same hart and its handler attempted `acquire(&L)`, the handler would spin waiting for a lock owned by the interrupted execution. The interrupted execution cannot resume to release `L` until the handler returns. This is same-hart self-deadlock.

That is why LUIT's lock discipline disables local interrupts before taking the spinlock and restores the previous local state after release.

Now consider nesting:

```
push_off(); // depth 1, interrupts off
push_off(); // depth 2, still off
pop_off(); // depth 1, must remain off
pop_off(); // depth 0, restore original state
```

A naive “disable in acquire, enable in every release” rule could enable interrupts when the inner lock is released even though an outer protected region is still active.

Checkpoint F. Explain why disabling interrupts on Hart 0 does not provide mutual exclusion against Hart 1. What provides that cross-hart exclusion instead?

11 Part H - Final regression and understanding check

End the debugging session and remove temporary GDB breakpoints. In the scratch branch, keep the added `user/pipe_stress.c` file and its Makefile `UPROGS` entry.

Rebuild from a clean state:

```
make clean
make
make grade
python3 tools/lab05_preflight.py
```

Then boot at least once with multiple harts and confirm:

```
pipe_stress
```

ends with:

```
PIPE_STRESS: PASS
```

Finish by checking that every statement below can be explained from source code or an observed execution rather than from memorization.

- ☐ I can draw a lost-update interleaving.
- ☐ I can explain why a lock needs atomic acquisition.
- ☐ I can distinguish lock ownership from memory ordering.
- ☐ I can explain why interrupt disabling is hart-local.
- ☐ I can explain the same-hart interrupt self-deadlock scenario.
- ☐ I can explain why `push_off()`/`pop_off()` track nesting.
- ☐ I can identify the predicate and protecting lock in the LUIT pipe path.
- ☐ I can explain when pipe writers/readers sleep and what wakes them.
- ☐ I can explain what a sleep channel is.
- ☐ I can construct the lost-wakeup schedule.
- ☐ I can explain why `sleep(chan, lock)` takes a lock.
- ☐ I can explain why a waiting condition is tested in a while loop.
- ☐ I can distinguish spinning from sleeping.
- ☐ I can run the same synchronization workload correctly with 1, 2, and 4 harts.

Reading connection

For the textbook view, revisit the OSTEP chapters on locks and condition variables. Map the abstract ideas to the LUIT mechanisms observed here:

General concept	LUIT mechanism observed here
atomic lock acquisition	compiler atomic RMW in <code>acquire()</code>
short critical-section waiting	spinlock
local interrupt discipline	<code>push_off()/pop_off()</code>
wait for shared predicate	<code>sleep(chan, lock)</code>
wake possible waiters	<code>wakeup(chan)</code>
re-check after wake	while around the condition
bounded producer/consumer state	kernel pipe buffer